

Nombre y Apellido	LU	Nº orden	# hojas
Tomas Rodriguez Nodda	431/23	21	6

Complejidad Computacional

Primer parcial - 1er Cuatrimestre 2025

9,3/10
Corrección LAUS

Duración : 4 horas

Este examen es a libro cerrado. Para aprobar se deben alcanzar 50 puntos, y tener por lo menos 15 puntos en el ejercicio 1) y 15 puntos en el ejercicio 3).

Ayuda para el parcial: Recordar que $\binom{a}{b} \leq a^b$.

1 (30 puntos) Decidir si las siguientes afirmaciones son verdaderas, falsas, o si implican la solución de alguna pregunta abierta. En cualquiera de los casos, demostrar.

- ✓ a) Si $P = NP$, entonces $coNP = NP$.
- ✓ b) $NPSPACE = coNPSPACE$.
- ✓ c) Si $P \neq NP$ entonces cualquier problema en NP que no esté en P debe ser NP -hard.
- ✓ d) Todos los problemas incluidos en NP (salvo los triviales¹) son NP -completos.
- ✓ e) SAT es $PSPACE$ -hard.
- ✓ f) Existe un $k \in \mathbb{N}$ tal que $NP \subseteq NTIME[n^k]$.

Observación: No se da puntaje a respuestas sin justificar.

2 (20 puntos) Enunciar y demostrar el teorema de jerarquía de tiempos determinísticos.

3 (30 puntos) Considerar los siguientes problemas:

- $BALANCED-3-SAT = \{(\varphi) : \varphi \text{ es una fórmula 3CNF satisfacible por una asignación que manda tantas variables a falso como a verdadero}\}$
- $AT-MOST-3-SAT = \{(\varphi) : \varphi \text{ es una fórmula 3CNF y hay una asignación que satisface la fórmula que manda a lo sumo 10 variables a verdadero}\}$

Para cada uno decidir si está en P o si es NP -completo². En cualquiera de los casos, demostrar la afirmación.

4 (20 puntos) Dado un lenguaje Π y un número natural $k \in \mathbb{N}$ definimos

$$\Pi^k = \{x_1 x_2 \dots x_k : x_i \in \Pi, \forall 1 \leq i \leq k\}$$

$$\text{y } \Pi^* = \bigcup_{k \in \mathbb{N}} \Pi^k.$$

- a) Probar³ que para todo $k \in \mathbb{N}$ vale que si $\Pi \in P$ entonces $\Pi^k \in P$.
- b) Probar que si $\Pi \in NP$ entonces $\Pi^* \in NP$.

¹Los lenguajes triviales son el vacío y el que contiene a todas las cadenas

²En este ejercicio asumimos que $P \neq NP$.

³En caso de ser necesario se puede asumir en este y en el siguiente inciso que $\lambda \notin \Pi$, donde λ es la cadena vacía (por más que el resultado sea independiente de esta suposición).

Tomas
Rodríguez Rodin

LV: 431/23

Hojas

1) a) Verdadero. Porque P está cerrado por complemento.
 $\cdot NP \subseteq CoNP$

Tomemos $L \in NP$, por hipótesis $L \in P$. Aunque
hay una máquina determinística M que computa L
en tiempo polinomial, o decir

$$M(X) \in L \Leftrightarrow M(X) = 1$$

Luego podemos crear M' donde

$M'(X)$: - calcula $M(X)$ (polinomial)

- devuelve $1 - M(X) = 0(1)$

Luego $M'(X)$ es polinomial $\wedge L(M') = L^c$

$$\Rightarrow L^c \in P = NP \Rightarrow L \in CoNP$$

$\cdot CoNP \subseteq NP$

Tomemos $L \in CoNP$.

Luego $L^c \in NP$, por el punto anterior $L^c \in CoNP$

$\Rightarrow L \in NP$

$$\therefore NP = CoNP$$

b) Verdadero.

Tomemos $L \in PSPACE$. Luego $\exists N$ una máquina no
determinística T_N

$$X \in L \Leftrightarrow \exists \text{ un cómputo aceptador de } T_N(X)$$

$\wedge T_N(X)$ computa en un ~~espacio~~ ^{espacio} polinomial $S(n)$

podemos crear una máquina determinística M que
~~simula todos los cómputos~~ con entrada X simula ~~un~~ todas

(a) ~~cómputos~~ ^{ejecuciones} de T_N con entrada X en orden $\wedge$ reutilizando
espacio. Rechaza si encuentra un cómputo aceptador.

Al reutilizar espacio, usa $S(n)$ en espacio. $\wedge$

$$L(M) = L^c, \therefore L^c \in PSPACE$$

Recordemos que por Teorema de SAVITCH

$$PSPACE = NPSPACE, \text{ luego } L^c \in NPSPACE.$$

Si tomamos $L^c \in coNPSPACE$, podemos tomar $L^c \in NPSPACE$
 $\wedge$ repetir el proceso para ver que $L \in NPSPACE$.

$$\therefore NPSPACE = coNPSPACE.$$

c) FALSO.

El teorema de Ladner dice

$$P \neq NP \Rightarrow \exists L \in NP \text{ t.q. } L \notin P \text{ y } L \notin NP\text{-hard}$$

Esta afirmación contradice ~~el~~ el teorema de Ladner, porque dice que $\nexists L \in NP \text{ t.q. } L \notin P \wedge L \notin NP\text{-hard}$
o $NP \setminus (P \cup NP\text{-hard}) = \emptyset$

Por lo tanto debe ser falso. ✓

d) Es una pregunta abierta, porque $P \subseteq NP$,

si fuera verdadero, cualquier $L \in P$ sería NP -completo
ya que $\forall L' \in NP$, $L' \leq_P L$ y esto implica que $L' \in P$ y
 $NP = P$, que es una pregunta abierta como vimos en
la materia. ✓

~~El problema P vs NP~~

~~El problema P vs NP es el más importante de la informática~~

~~porque es el más importante de la informática~~

~~porque es el más importante de la informática~~

Hago primero el F porque el R no me está saliendo (No sé)

e) Falso. Supongamos que $NP \subseteq NTIME(n^k)$, $k \in \mathbb{N}$.

Notar que n^k y n^{k+1} son constructibles en tiempo

$$(n+1)^k = O(n^{k+1})$$

$$\Rightarrow NTIME(n^k) \subsetneq NTIME(n^{k+1}) \text{ por teorema de}$$

la jerarquía de tiempos no determinísticos

$$\text{y } NTIME(n^{k+1}) \subseteq NP \text{ por (1)}$$

$$NP = \bigcup_{c \in \mathbb{N}} NTIME(n^c), \text{ en particular } c = k+1$$

$$\text{Luego } NTIME(n^{k+1}) \subseteq NP \subseteq NTIME(n^k)$$

$$\Rightarrow NTIME(n^{k+1}) = NTIME(n^k)$$

Pero esto contradice el teorema de jerarquía de
tiempos no determinísticos ABS! ✓

21.

$$f(n) \mid o_3(f(n)) = o(g(n))$$

Demoi: Sea m_x la x -ésima muestra determinística.

contra X:

- $$O(3(|x|))$$

Suponhamos $\exists L(D) \in \partial T \cap M_E(F(n))$, $c = 10 - \epsilon c$

mitte groß $\times L(m) = L(0)$.

$$-M = M_i \quad -C(F(1)) \log(C(F(1))) < 3 \log(1)$$

- $m(i)$ computa en tiempo $C \cdot f(i)$

$U(i, j)$ computa en tiempo $C(f(n) \log(C(f(n))))$

$$C(f(n) \log(f(n))) \leq D(f(n) \log(f(n))) < E(n)$$

$$v(i, i) \text{ terms are } \leq 3(l_i) \text{ prob.}$$
$$D(i) = 1 - m_i(i) = 1 - m(i)$$

(para todos) $L(n) = L(m)$ ABS!

3) AT-MOST-3-SATEP:

Recordemos que dado v una valoración, decidir si $v \models \varphi$ es polinomial con respecto a $|\varphi|$.

Como podemos marcar a la suma de variables como verdadera, solo debemos ~~probar las valoraciones~~ crear / probar las valoraciones con la suma de variables verdaderas, y la cantidad de valoraciones es

$$\binom{n}{0} + \binom{n}{1} + \dots + \binom{n}{n} = 1 + n + \dots + n^n + n^{n^0} = O(n^{n^0})$$

posibles valoraciones con $n = |\varphi|$

Si escribimos cada valoración como un bitmap de $|\varphi|$ bits, crear cada valoración es $O(n)$.

Luego el costo de este programa sería
 $O(n^{n^0}) \times O(n) \times O(F(n)) = O(n^{n^0} F(n))$ con F polinomial

↑ ↑ ↑
 cantidad de valoraciones creando valoración evaluar una valoración

Donde $F(n)$ es polinomial.

∴ AT-MOST-3-SATEP

• BALANCED-3-SATEP es $\mathcal{NP}$ -completo.

~~Se debe~~

→ BALANCED-3-SAT (B-3-SAT) es $\mathcal{NP}$ -completo si B-3-SAT es $\mathcal{NP}$ y B-3-SAT es $\mathcal{NP}$ -hard

→ $\mathcal{NP}$ creamos una máquina determinística M .

pasamos como certificado la valoración v como un bitmap sea $\varphi \in \text{B-3-SAT}$, $v \in \{0,1\}^{|\varphi|}$. Luego vemos que v tenga igual cantidad de 1's y 0's $O(|\varphi|)$ y luego vemos que $v \models \varphi$, que es polinomial.

si no del otro lado

- NP-hard: $\forall \epsilon > 0, 3\text{-SAT} \leq_p B\text{-}3\text{-SAT}$.

Es decir, $\exists$ existe f polinomial t.c.

$$X \in 3\text{-SAT} \Leftrightarrow f(X) \in B\text{-}3\text{-SAT}$$

Tomemos

ϕ una fórmula, con variables $x_1, x_2, \dots, x_n$, $n \leq | \phi |$, $n \in \mathbb{N}$.

Creo las variables $z_1, z_2, \dots, z_n$

Asumo $\phi \in 3\text{-CNF}$, caso contrario lo rechazamos.
Cada cláusula C_j de ϕ tiene la forma

$$(J_1(x_a) \vee J_2(x_b) \vee J_3(x_c)) \text{ con } 1 \leq a, b, c \leq n$$

$$\text{y cada } J(x_i) = \begin{cases} x_i & \text{cuálquiera de los 2.} \\ \neg x_i \end{cases}$$

con $f(\phi)$ por cada C_j creo 4 cláusulas:

$$(J_1(x_a) \vee J_2(x_b) \vee J_3(x_c)) \wedge (\neg J_1(z_a) \vee J_2(x_b) \vee J_3(x_c)) \wedge \\ (J_1(x_a) \vee \neg J_2(z_b) \vee J_3(x_c)) \wedge (J_1(x_a) \vee J_2(x_b) \vee \neg J_3(z_c))$$

$f(\phi)$ es polinomial, porque todo lo que hace es crear 4 cláusulas por cada cláusula de ϕ , así que es lineal.

Notar que

$$\neg J_i(z_a) = \neg \neg z_a \quad \text{si } J_i(x_a) = x_a \\ = \neg z_a \quad \text{si } J_i(x_a) = \neg x_a$$

Ahora vemos que

$$\phi \in 3\text{-SAT} \Leftrightarrow f(\phi) \in B\text{-}3\text{-SAT}$$

$$\Rightarrow) \text{ Asumo } f(\phi) \in B\text{-}3\text{-SAT}.$$

En particular, todas las cláusulas de ϕ están en $f(\phi)$
así que todas ellas deben ser verdaderas para que $f(\phi)$
sea satisficible. Y todas las cláusulas tienen 3 literales
 $\therefore \phi \in 3\text{-SAT}$

$$\Rightarrow) \text{ Asumo } \phi \in 3\text{-SAT}$$

Luego hay una valoración v t.c. $v \models \phi$

que podemos extender esta valoración a

$$v \models z_i \Leftrightarrow v \models x_i \quad \forall 1 \leq i \leq n \text{ con } n \text{ la cantidad de variables de } \phi$$

Tema 3

Lvi 431/23

Hoja 9

Rodriguez Nodda

Notar que $\forall x_i$, para cualquier cláusula de la forma (sin importar orden)

$(x_i \vee L_2 \vee L_3)$ o $(\neg x_i \vee L_2 \vee L_3)$ en φ , $f(\varphi)$ crea una cláusula

$(\neg z_i \vee L_2 \vee L_3)$ y para toda cláusula de la forma (sin importar orden)

$(\neg x_i \vee L_2 \vee L_3)$ hay una cláusula

$(z_i \vee L_2 \vee L_3)$

Si tomamos $v \models z_i \Leftrightarrow v \not\models x_i$, podemos reemplazar z_i por $\neg x_i$, luego

~~$(\neg \neg z_i \vee L_2 \vee L_3) = (z_i \vee L_2 \vee L_3)$~~

$(\neg z_i \vee L_2 \vee L_3) \equiv (\neg \neg x_i \vee L_2 \vee L_3) \equiv (x_i \vee L_2 \vee L_3)$ en φ

$(z_i \vee L_2 \vee L_3) \equiv (\neg x_i \vee L_2 \vee L_3)$ en φ

Como estas cláusulas están en φ , sabemos que v (al menos como verdadera), luego si $v \models z_i \Leftrightarrow v \not\models x_i$, también tomamos como verdaderas las de $f(\varphi)$ por lo que $f(\varphi)$ es satisfacible, con esta extensión de v .

Veamos que esto es balanceado.

$\forall (z_i \leq n) \quad v \models z_i \Leftrightarrow v \not\models x_i$, es decir para cada x_i

falso, z_i es verdadero y para cada x_i verdadero, z_i es falso,

por lo tanto hay tantas variables verdaderas como falsas

$\therefore f(\varphi) \in B-3-SAT$

y $B-3-SAT \in NP$ -completo

Revisando si que la función podía transformar la cláusula

$(J_1(x_a) \vee J_2(x_b) \vee J_3(x_c))$ en 2 cláusulas

$(J_1(x_a) \vee J_2(x_b) \vee J_3(x_c)) \wedge (\neg J_1(z_a) \vee \neg J_2(z_b) \vee \neg J_3(z_c))$

y sigue valiendo todo lo que dije "

$J \in JE$, PERO ESTÁ BIEN IGUAL

Toma)

201931/23

Hoja 5

Rodriguez Nadin

4) 2) prueba por inducción sobre k :

$P(k): \pi \in P \Rightarrow \pi^k \in P$

• caso base: $k=1$

Es trivial, pues $\pi^1 = \pi \in P$

• caso inductivo: Asumo que vale para k y que vale para $k+1$.

Llamo γ a la entrada y llamo

$\gamma(i:b)$ los elementos de γ desde el dígito i hasta el b inclusive, si $b < i$, es λ .

Luego podemos crear un algoritmo para $\pi^{k+1}(\gamma)$:

For $i := 0, 1, \dots, |\gamma|$:

if $(\gamma(1:i) \in \pi \wedge \gamma(i+1:|\gamma|) \in \pi^k)$

return 1

fi

end for

return 0

~~Supongamos~~ Por hipótesis, $\pi \in P \wedge \pi^k \in P$, digamos,

decidir $x \in \pi$ toma tiempo $c|x|^c$ con $c \in \mathbb{N}$ y

decidir $x \in \pi^k$ toma tiempo $d|x|^d$ con $d \in \mathbb{N}$.

Luego este algoritmo tiene en

Como $|\gamma(i:b)| \leq |\gamma|$, este algoritmo es

$O(|\gamma|(c|\gamma|^c + d|\gamma|^d)) = O(|\gamma|^{c+1} + |\gamma|^{d+1})$ que es polinomial

~~en π^{k+1}~~

$O(|\gamma|^{c+1} + |\gamma|^{d+1})$

~~b) $\pi \in P$ luego $\exists$ una máquina determinista T_π~~

~~$x \in \pi \Leftrightarrow \exists u \in \{0,1\}^{P(|x|)} T_\pi(x,u) = 1$~~

~~T_π computa en tiempo polinomial respecto al tamaño de la entrada, y P es polinomial.~~

~~Que $\exists$ una máquina determinista~~

Me faltó justificar más el 2, sigue el otro lado

si se cumple $z=1$

$y(i,i) \in \Pi$ y adem s $y(i+1, |y|) \in \Pi^K$, si $z=1$ es
 $z=1 \Rightarrow x_1 = y(i,i)$ y $z=1$

$\exists x_2, x_3, \dots, x_{K+1} \in \Sigma$ tal que $x_2 x_3 \dots x_{K+1} = y(i+1, |y|)$ donde

$x_i \in \Pi$, por lo que hay $K+1$ elementos de Π en y .

Como el algoritmo es ~~no determinista~~ polinomial y

$y \in \Pi^{K+1} \Leftrightarrow M(y)=1$ con M m quina que computa el
algoritmo, $\Pi^{K+1} \in P$.

$\therefore \Pi^K \in P \quad \forall K \in \mathbb{N}$. ✓

b) $\Pi \in NP$, luego $\exists$ M quina no determinista tal

que computa en tiempo polinomial respecto a la entrada

$x \in \Pi \Leftrightarrow \exists v \in \{0,1\}^{p(|x|)} \quad M(x,v)=1$ donde p es un polinomio.

As  $\exists M'$ que cumple las mismas condiciones para Π^K $z=1$
en Π .

Consideremos la cadena

$y = x_1 x_2 \dots x_K$ con K no fijo $K \in \mathbb{N}$.

Podemos crear un certificado

~~$v = \langle a_1, b_1, u_1 \rangle, \langle a_2, b_2, u_2 \rangle, \dots, \langle a_K, b_K, u_K \rangle$~~

donde la tripleta ~~$\langle a_i, b_i, u_i \rangle$~~

$v = \langle \text{fin}_1, u_1 \rangle, \langle \text{fin}_2, u_2 \rangle, \dots, \langle \text{fin}_K, u_K \rangle$, posici n de la

donde la dupla $\langle \text{fin}_i, u_i \rangle$ nos indica el  ltimo c falo de
 x_i en y el certificado tal que $M(x_i, u_i)=1$

Notar que no es necesario pasar el inicio de x_i , porque

inicio de $x_0 = 1$ y inicio de $x_{i+1} = \text{fin}_i + 1$

$\forall 1 \leq i \leq K$

vamos que tiene tama o polinomial:

• $|\text{fin}_i| \leq |y|$

• $|u_i| = p(|x_i|) \leq p(|\text{fin}_i|) \leq p(|y|)$

Luego

$|v| \leq K^k |y|^k p(|y|)$ y $K \leq |y|$ porque asumimos $\lambda \notin \Pi$,

luego $|x_i| > 0$ y hay al menos $|y|$ x . Notar que $|y|^k p(|y|)$ es un polinomio

si $z=1$ en otro Lema.

Ahora hay que definir un algoritmo para m' :

Primero verifico $|y| \leq |x|^2 p(|x|)$

$|y| \leq |x|^2 p(|x|)$ caso contrario rechazo. $O(|x|^2 p(|x|))$

Luego tomo cada par $\langle f_{in_i}, u_i \rangle$ y computo

$m(y(f_{in_i} \upharpoonright f_{in_i}) u_i)$ si $i > 1$

$m(y(f_{in_i}) u_i)$ si $i = 1$,

~~si~~ notando que si $\exists 1 \leq i \leq K-1$ t.z.

$f_{in_{i+1}} < f_{in_i} \Rightarrow y(f_{in_i} \upharpoonright f_{in_{i+1}}) = \lambda$, y como $\lambda \notin \Pi$,
rechazo siempre.

~~si $m(y(f_{in_i}) u_i) = 0$ para algún i rechazo~~

si $m(y(f_{in_i}) u_i) = 0$ para alguno de estos simulaciones,
lo rechazo.

si todos devuelven 1, entonces devuelvo 1.

no lo dije antes, pero chequeo $f_{in_K} \geq |y|$

Digo que m computa en tiempo $c n^c$ para algún $c \in \mathbb{N}$,

luego cada simulación toma tiempo 2 lo sumo $c(|y| p(|y|))^c$ y como

y como lo hace 2 lo sumo $|y|$ veces,

m' computa en tiempo $O(c |y| (|y| p(|y|))^c)$

$= O(|y|^{c+1} p(|y|)^c)$ es m' es polinomial.

Ahora vemos que m' computa Π^K

$y \in \Pi^K \Leftrightarrow y = x_1 x_2 \dots x_K : x_i \in \Pi \forall 1 \leq i \leq K, K \in \mathbb{N}$

$\Leftrightarrow y = x_1 x_2 \dots x_K : K \in \mathbb{N}, \forall 1 \leq i \leq K, \exists u_i \in \{0,1\}^{|x_i|} m(x_i u_i) = 1$

~~$\Leftrightarrow y = x_1 x_2 \dots x_K : K \in \mathbb{N} m'(y) = 1$~~

$\Leftrightarrow K \in \mathbb{N}, \exists u = \langle f_{in_1}, u_1 \rangle, \langle f_{in_2}, u_2 \rangle, \dots, \langle f_{in_K}, u_K \rangle$

donde $f_{in_K} = |y|$

$m(y(f_{in_1}) u_1) = 1$

$\forall 1 < i \leq K m(y(f_{in_i} \upharpoonright f_{in_i}) u_i) = 1$

$\Leftrightarrow \exists u \in \{0,1\}^{|y| p(|y|)} m'(y u) = 1$

$\therefore \Pi^K \in NP$